

CLASS ACTIVITY - 2

Process

03-10-2024

23BAI10917
VIJAY RAJESH R

Questions:

1. Difference between Process and Threads.
2. State the states of a process
3. Five important points regarding Context Switching.

Answers:

1. Difference between Process and Threads:

Definition:

Process: A process is an independent program in execution, containing its own memory space.

Thread: A thread is a smaller unit of a process that can run concurrently, sharing the process's memory space.

Memory:

Process: Has its own memory space, including code, data, and system resources.

Thread: Shares the process's memory and resources, but has its own stack and registers.

Overhead:

Process: Creation and management involve more overhead due to isolation and resource allocation.

Thread: Lighter and faster to create and manage, as they share resources.

Communication:

Process: Inter-process communication (IPC) is required, which can be complex.

Thread: Threads can communicate more easily through shared memory.

Isolation:

Process: More isolated; an error in one process does not affect others.

Thread: Less isolated; an error in one thread can affect the entire process.

2. States of a Process:

- New: The process is being created.
- Ready: The process is waiting to be assigned to a processor.
- Running: The process is currently being executed by the CPU.
- Waiting: The process is waiting for some event (like I/O completion).
- Terminated: The process has finished execution and is being removed from memory.

3. Five Important Points Regarding Context Switching:

- Definition: Context switching is the process of storing the state of a currently running process or thread, so it can be resumed later, and loading the state of the next process or thread to run.
- Overhead: Context switching incurs overhead due to the time taken to save and load states, which can impact overall system performance, especially if done excessively.
- State Saving: During a context switch, the CPU registers, program counter, and other process-specific data must be saved and restored, which is critical for proper process execution.
- Latency: Context switching introduces latency, which can affect the responsiveness of applications, particularly in systems with a high number of processes or threads.

- Multitasking: Efficient context switching is essential for multitasking operating systems, as it enables them to share CPU time among multiple processes, allowing for concurrent execution.